

BB AD DBMS Written

কীভাবে ভাবব? কীভাবে **Approach** করব? কীভাবে **Practice** করব?

Theory থেকে Real Exam — সম্পূর্ণ বাংলায় Mental Framework

সমস্যা	বাস্তবতা	সমাধান
Theory পড়েছ কিন্তু পরীক্ষায় answer করতে পারছ না	BB AD Written সরাসরি definition জিজ্ঞেস করে না — scenario দিয়ে APPLY করতে বলে	Thinking pattern শেখো, শুধু content না

এই guide-এ BB AD-এর সব DBMS written question-এর exact thinking process বাংলায় বোঝানো হয়েছে।

কেন Theory পড়েও Answer করা যায় না?

BB AD DBMS Written-এ fail করার সবচেয়ে বড় কারণ হলো: student theory মুখস্থ করে কিন্তু সেটা apply করতে পারে না। নিচের comparison দেখো:

Theory যা বলে	BB AD Written যা জিজ্ঞেস করে	Difference কোথায়
Normalization মানে data redundancy কমানো	Rank(Br_Name, Br_City, Assets, Acc_name, Acc_Num, Balance) — এই schema normalize করো এবং PK, FK দেখাও	Schema দেখে নিজে FD বের করে 3NF-এ আনতে হবে
SQL-এ SELECT, FROM, WHERE, GROUP BY ব্যবহার হয়	Movies, HasRole, HasGenre table থেকে romantic comedy movies-এর count বের করো	Real table structure দেখে সঠিক JOIN logic বের করতে হবে
ACID মানে Atomicity, Consistency, Isolation, Durability	A থেকে B-তে balance transfer হলো না — কোন ACID property দায়ী?	Scenario পড়ে কোন property লক্ষিত হয়েছে সেটা identify করতে হবে
Primary Key সারি uniquely identify করে	Employee(Empid,Firstname), Works(Empid,salary), Manager — employee count per dept query লেখো	Table structure বুঝে সঠিক JOIN + GROUP BY logic লিখতে হবে

কীভাবে ভাববেন:

BB AD test করে: তুমি concept USE করতে পারো কিনা। Definition repeat করলে marks পাবে না। প্রতিটা concept পড়ার পরে নিজেকে জিজ্ঞেস করো — "যদি একটা নতুন schema বা scenario দেওয়া হয়, আমি কি এটা apply করতে পারব?"

01 DBMS Written-এর ৫টা Type — চিনে নিলে অর্ধেক কাজ শেষ

BB AD ও Other Banks-এর সব DBMS written question মাত্র ৫টা type-এ পড়ে। একবার type চিনলে exact approach জানা যায়।

Type	Trigger Words (question-এ এই কথা থাকে)	তুমি কী করবে	কোন বছর এসেছে
Type 1: Normalization	normalize করো, 3NF-এ আনো, FD বের করো, schema decompose করো	FD identify → NF check → decompose → PK/FK দেখাও	BB AD 2025, Other 2022, 2024
Type 2: SQL Query লেখো	SQL query লেখো, find করো, count করো, UPDATE করো, JOIN করো	Table structure বোঝো → কোন JOIN? → GROUP BY? → HAVING? → query লেখো	BB AP 2023, Other 2020-2026
Type 3: ACID Scenario	কোন property লঙ্ঘিত? কোন property দায়ী? ACID explain করো	Scenario পড়ো → কী ভুল হয়েছে identify করো → কোন property এটা prevent করত	BB AD 2025, Other 2022, 2024
Type 4: ER Diagram Design	ER diagram draw করো, entity-relationship design করো	Text থেকে entity বের করো → relationship identify করো → cardinality ঠিক করো → draw করো	BB AP 2023, Other 2020-2022
Type 5: Short Note/Comparison	পার্থক্য বলো, short note লেখো, DBA-র কাজ কী	Definition → How it works → Example — এই ৩ ভাগে লেখো	BB AP 2019, Other 2021-2024

BB NOTE: Type 1 (Normalization) এবং Type 2 (SQL) — এই দুটো থেকেই BB AD Written-এ সবচেয়ে বেশি প্রশ্ন আসে। এই দুটো আগে master করো।

02 TYPE 1 — Normalization প্রশ্ন (সবচেয়ে গুরুত্বপূর্ণ)

Normalization প্রশ্নে তোমাকে একটা schema দেওয়া হয় এবং বলা হয় সেটা normalize করতে। এটা pure memorization দিয়ে হবে না — তোমাকে step-by-step analyze করতে হবে।

Normalization প্রশ্ন দেখলে মাথায় যা আসবে

কীভাবে ভাববেন: Normalization প্রশ্নে সবসময় ৫টা ধাপে ভাবো: ১. এই schema-তে কোন attributes আছে? ২. কোনটা কোনটা determine করে? (FD বের করো) ৩. কোনটা Primary Key হতে পারে? ৪. কোনো Partial FD আছে? (2NF ভাঙছে?) ৫. কোনো Transitive FD আছে? (3NF ভাঙছে?) তারপর decompose করো।

Functional Dependency (FD) কীভাবে identify করবে

FD মানে: $X \rightarrow Y$ মানে X জানলে Y জানা যায়।

Schema দেখলে নিজেকে জিজ্ঞেস করো: "এই column-এর value জানলে কি অন্য কোনো column-এর value automatically জানা যায়?"

যদি দেখো...	তাহলে FD হলো...	কারণ
Acc_Num (Account Number)	$\text{Acc_Num} \rightarrow \text{Acc_name}, \text{Br_Name}, \text{Balance}$	একটা account number দিলে customer name, branch, balance সব জানা যায়
Br_Name (Branch Name)	$\text{Br_Name} \rightarrow \text{Br_City}, \text{Assets}$	একটা branch name দিলে সেই branch-এর city এবং assets জানা যায়
StudentID	$\text{StudentID} \rightarrow \text{StudentName}, \text{DateOfBirth}$	Student ID দিলে name এবং DOB জানা যায়
DeptID	$\text{DeptID} \rightarrow \text{DeptName}, \text{Location}$	Department ID দিলে name এবং location জানা যায়

৩টা Normal Form — সহজ ভাষায়

Normal Form	Rule (এক কথায়)	এটা ভাঙলে কী হয়	Fix কীভাবে
1NF	প্রতিটা cell-এ একটাই value থাকবে — list বা group থাকবে না	Data inconsistency — একই data multiple জায়গায় update করতে হয়	Multi-valued attribute-কে আলাদা table-এ নাও
2NF	Composite PK থাকলে, সব non-key attribute-কে পুরো PK-এর উপর depend করতে হবে — PK-এর অংশের উপর না	Partial dependency — Update anomaly হয়	Partial FD-কে আলাদা table-এ নাও
3NF	Non-key attribute অন্য non-key attribute-এর উপর depend করতে পারবে না (transitive FD নেই)	Transitive dependency — এক জায়গায় update করলে inconsistency	Transitive FD-কে আলাদা table-এ নাও

৫-ধাপের Normalization Method — সবসময় এই order follow করো

ধাপ ১: Schema-র সব attribute লিস্ট করো

ধাপ ২: FD (Functional Dependencies) identify করো — $X \rightarrow Y$ pattern

ধাপ ৩: Primary Key কোনটা? (সবচেয়ে বেশি কিছু determine করে)

ধাপ ৪: 1NF check $\rightarrow$ 2NF check $\rightarrow$ 3NF check — কোথায় problem?

ধাপ ৫: Problem table-কে decompose করো, PK এবং FK দেখাও

QUESTION: BB AD 2025 Written Q6 (Exact): Schema: Rank(Br_Name, Br_City, Assets, Acc_name, Acc_Num, Balance)(a) Normalize করো এবং Primary Key ও Foreign Key দেখাও(b) নতুন schema কেন ভালো form-এ আছে তা explain করো

কীভাবে ভাববেন:

প্রথমে ভাবো: এই ৬টা attribute-এর মধ্যে কে কাকে determine করে?

Acc_Num দিলে কি সব জানা যায়?

না — Acc_Num দিলে জানা যায় Acc_name, Balance, এবং Br_Name।

কিন্তু Br_Name দিলে জানা যায় Br_City এবং Assets।

তাহলে $Acc_Num \rightarrow Br_Name \rightarrow Br_City$ মানে Acc_Num থেকে Br_City পাওয়া যাচ্ছে পরোক্ষভাবে।

এটাই Transitive Dependency — 3NF ভাঙছে!

MODEL ANSWER — BB AD 2025 — Schema Normalization (Step-by-Step)

দেওয়া Schema: Rank(Br_Name, Br_City, Assets, Acc_name, Acc_Num, Balance)

ধাপ ১ — FD (Functional Dependencies) identify করো:

FD1: $Acc_Num \rightarrow Acc_name, Br_Name, Balance$ (Account number দিলে customer name, branch, balance জানা যায়)

FD2: $Br_Name \rightarrow Br_City, Assets$ (Branch name দিলে city এবং assets জানা যায়)

ধাপ ২ — Primary Key নির্ধারণ:

Acc_Num সবচেয়ে বেশি কিছু determine করে $\rightarrow$ Primary Key = Acc_Num

ধাপ ৩ — 1NF Check:

সব attribute atomic (একটাই value) $\rightarrow$ 1NF-এ আছে ✓

ধাপ ৪ — 2NF Check:

PK single attribute (Acc_Num) $\rightarrow$ Partial FD সম্ভব নয় $\rightarrow$ 2NF-এ আছে

✓ ধাপ ৫ — 3NF Check:

$Acc_Num \rightarrow Br_Name$ (direct) $Br_Name \rightarrow Br_City, Assets$ তাহলে: $Acc_Num \rightarrow Br_City$ (TRANSITIVE!) $\leftarrow$ 3NF ভাঙছে!
Br_Name non-key attribute কিন্তু Br_City এবং Assets determine করছে।

ধাপ ৬ — 3NF-এ Decompose করো:

Table 1: Branch(Br_Name [PK], Br_City, Assets) Table 2: Account(Acc_Num [PK], Acc_name, Balance, Br_Name [FK $\rightarrow$ Branch])

ধাপ ৭ — Verify: Branch table: $Br_Name \rightarrow Br_City, Assets$. Br_Name PK. Transitive FD নেই. 3NF ✓ Account table: $Acc_Num \rightarrow Acc_name, Balance, Br_Name$. Acc_Num PK. Transitive FD নেই. 3NF ✓

উত্তর (b) — কেন নতুন schema ভালো:

Original schema 3NF-এ ছিল না কারণ Br_Name (non-key) থেকে Br_City, Assets (non-key) নির্ধারিত হচ্ছিল। Decompose করার পরে প্রতিটা table-এ প্রতিটা non-key attribute শুধু PK-এর উপর নির্ভর করছে। তাই নতুন schema 3NF-এ আছে — redundancy নেই, update/delete anomaly নেই।

PRACTICE: Normalization Practice: প্রতিদিন ১টা নতুন schema নাও এবং নিজে ৫ ধাপে normalize করো: ১. Student(StudentID, StudentName, CourseID, CourseName, InstructorID, InstructorName) ২. Order(OrderID, CustomerID, CustomerName, ProductID, ProductName, Quantity, Price) ৩. Employee(EmpID, EmpName, DeptID, DeptName, DeptLocation, ManagerID, ManagerName) Answer check করো Claude-এর সাথে।

03 TYPE 2 — SQL Query লেখা (দ্বিতীয় সবচেয়ে কমন)

SQL প্রশ্নে তোমাকে table structure দেওয়া হয় এবং একটা information বের করতে বলা হয়। এখানে syntax মুখস্থ করলেই হবে না — logic বুঝতে হবে।

SQL প্রশ্ন দেখলে ৪টা জিনিস বের করো

কীভাবে ভাববেন: SQL প্রশ্ন পড়লে প্রথমে এই ৪টা জিনিস identify করো:

১. কোন TABLE(s) থেকে data নেব? (FROM)
২. কোন condition-এ? (WHERE / HAVING)
৩. কোন COLUMN(s) দেখাব? (SELECT)
৪. কি GROUP করতে হবে বা SORT করতে হবে? (GROUP BY / ORDER BY) এই ৪টা ঠিক করলেই query লেখা সহজ হয়ে যায়।

SQL Execution Order — এটা না জানলে query লিখতে পারবে না

SQL query-এ যে order-এ কমান্ড লেখা হয় সেই order-এ execute হয় না। Execution order জানা না থাকলে WHERE vs HAVING-এ ভুল হয়।

Execute Order	Clause	কাজ কী	Example
১ম	FROM	কোন table(s) থেকে data নেব, JOIN করব	FROM Employee JOIN Dept
২য়	WHERE	Row-level filter — GROUP BY-র আগে	WHERE salary > 50000
৩য়	GROUP BY	কোন column দিয়ে group করব	GROUP BY department
৪র্থ	HAVING	Group-level filter — GROUP BY-র পরে	HAVING COUNT(*) > 5
৫ম	SELECT	কোন columns দেখাব, aggregate করব	SELECT dept, AVG(salary)
৬ষ্ঠ	ORDER BY	sort করব	ORDER BY avg_salary DESC

BB NOTE: BB MCQ 2025: "SQL query-এ কোন clause সবার আগে execute হয়?" উত্তর: FROM। SELECT নয়!

JOIN Types — কখন কোনটা ব্যবহার করব

JOIN Type	কখন ব্যবহার করব	Example scenario
INNER JOIN	দুটো table-এর matching rows চাই	Employee যাদের Department আছে তাদের দেখাও
LEFT JOIN	Left table-এর সব rows চাই, right-এ না থাকলে NULL	সব employee দেখাও — department না থাকলেও
RIGHT JOIN	Right table-এর সব rows চাই	সব department দেখাও — employee না থাকলেও
FULL JOIN	দুই table-এর সব rows চাই	Employee এবং Department সব দেখাও

QUESTION: BB AP 2023: Movies(mid,title,year), People(pid,name), Genres(gid,genre), HasRole(pid,mid,role), HasGenre(gid,mid) — romantic comedy movies-এর count বের করো।

কীভাবে ভাববেন: প্রথমে ভাবো: romantic comedy মানে একটা movie-কে দুটো genre-এ থাকতে হবে — Romance AND Comedy। কোন table-এ genre আছে? Genres এবং HasGenre। কোন table-এ movie আছে? Movies। তাহলে আমাকে Movies-কে HasGenre-এর সাথে এবং Genres-এর সাথে join করতে হবে — দুবার, দুটো genre-এর জন্য।

MODEL ANSWER — BB AP 2023 — Romantic Comedy SQL

Approach:

একটা movie romantic comedy হতে হলে সেটাকে HasGenre-এ দুটো entry থাকতে হবে — একটা Romance genre-এর জন্য, আরেকটা Comedy genre-এর জন্য।

```
SQL: SELECT COUNT(*) AS romantic_comedy_count
FROM Movies m
WHERE EXISTS
(
  SELECT 1 FROM HasGenre hg1 JOIN Genres g1 ON hg1.gid = g1.gid
  WHERE hg1.mid = m.mid AND g1.genre = 'Romance' )
AND EXISTS (
  SELECT 1 FROM HasGenre hg2 JOIN Genres g2 ON hg2.gid = g2.gid
  WHERE hg2.mid = m.mid AND g2.genre = 'Comedy' );
```

ব্যাখ্যা:

EXISTS subquery check করে:

এই movie-র জন্য Romance genre entry আছে?

দ্বিতীয় EXISTS check করে: এই movie-র জন্য Comedy genre entry আছে?

দুটোই true হলে সেই movie count হয়।

QUESTION: Other Bank 2022 SQL:

Worker(worker-id,worker-name,hourly-rate,skill-type), Assignment(worker-id,building-id,num-days), Building(building-id,building-type). Hospital building-এ কাজ করা workers-দের hourly rate 5% বাড়ায়।

কীভাবে ভাববেন: এটা UPDATE query। কোন workers update হবে? যারা Hospital building-এ assignment আছে। তাহলে Worker → Assignment → Building — ৩টা table জড়িত। UPDATE করতে WHERE-এ subquery ব্যবহার করব।

MODEL ANSWER — Other Bank 2022 — UPDATE with JOIN

```
UPDATE WorkerSET hourly_rate = hourly_rate * 1.05
WHERE worker_id IN (
  SELECT a.worker_id FROM Assignment a JOIN Building b ON a.building_id = b.building_id
  WHERE b.building_type = 'Hospital');
```

ব্যাখ্যা: Subquery Hospital building-এর সব worker_id বের করে। Main UPDATE এই worker_id-গুলোর hourly_rate 5% বাড়ায়।

QUESTION: BB AD 2016: Employee table থেকে second highest salary বের করো।

MODEL ANSWER — Second Highest Salary — দুটো উপায়

উপায় ১ — Subquery:

SELECT MAX(salary) FROM Employee WHERE salary < (SELECT MAX(salary) FROM Employee);

ব্যাখ্যা: সর্বোচ্চ salary বাদ দিয়ে বাকিদের মধ্যে সর্বোচ্চটা = second highest

উপায় ২ — LIMIT/OFFSET (MySQL): SELECT DISTINCT salary FROM Employee ORDER BY salary DESC LIMIT 1 OFFSET 1;

ব্যাখ্যা: Salary বড় থেকে ছোট sort করে, প্রথমটা skip করে (OFFSET 1), পরেরটা নাও

Aggregate Function + GROUP BY + HAVING — BB-তে সবচেয়ে বেশি আসা Pattern

এই pattern: "প্রতিটা [group] থেকে [aggregate] বের করো, যেখানে [condition]"

MODEL ANSWER — Template for GROUP BY + HAVING

Pattern: "প্রতিটা department-এ কতজন employee আছে — যেখানে ৫ জনের বেশি"

SELECT department, COUNT(*) AS emp_count FROM Employee GROUP BY department HAVING COUNT(*) > 5 ORDER BY emp_count DESC; Pattern:

"প্রতিটা department-এ average salary — যেখানে average > company average"

SELECT department, AVG(salary) AS avg_sal FROM Employee GROUP BY department HAVING AVG(salary) > (SELECT AVG(salary) FROM Employee) ORDER BY avg_sal DESC;

PRACTICE: SQL Practice: প্রতিদিন এই ৩টা query type লেখো from scratch:

১. একটা GROUP BY + HAVING query

২. একটা Multi-table JOIN query (Inner বা Left) ৩. একটা Subquery (WHERE ... IN (SELECT...)) Other bank-এর SQL questions থেকে নাও — Section 11 of DBMS notes।

04 TYPE 3 — ACID Scenario প্রশ্ন

ACID প্রশ্নে তোমাকে একটা scenario দেওয়া হয় এবং জিজ্ঞেস করা হয় কোন ACID property লঙ্ঘিত হয়েছে বা কোন property দায়ী। এখানে definition মুখস্থ করলেই হবে না — scenario পড়ে identify করতে হবে।

ACID Scenario Identification Framework

কীভাবে ভাববেন: ACID প্রশ্ন দেখলে scenario পড়ে এবং এই ৪টা প্রশ্ন করো:

১. Transaction কি অর্ধেক হয়ে গেছে? (Atomicity ভেঙেছে)
২. Data কি invalid state-এ আছে? Constraint ভেঙেছে? (Consistency ভেঙেছে)
৩. একটা transaction কি অন্যটার incomplete data দেখছে? (Isolation ভেঙেছে)
৪. Commit হওয়ার পরেও data হারিয়ে গেছে? (Durability ভেঙেছে)

Scenario পড়লে যা দেখবে	কোন ACID Property	মুখস্থের Shortcut
Transfer শুরু হলো কিন্তু মাঝপথে ব্যর্থ হলো — A থেকে টাকা কাটল কিন্তু B-তে পৌঁছালো না	Atomicity	ALL OR NOTHING — সব হবে বা কিছুই হবে না
Account balance negative হয়ে গেল — যা নিয়মবিরুদ্ধ	Consistency	VALID STATE — database সবসময় valid থাকবে
দুটো transaction একসাথে চলছে — একটা অন্যটার incomplete data দেখছে (Dirty Read)	Isolation	AS IF ALONE — প্রতিটা transaction একা চলছে মনে হবে
Power গেল — Commit করা data হারিয়ে গেল	Durability	PERMANENT — commit হলে চিরকালের জন্য

QUESTION: BB AD 2025 MCQ: A থেকে B-তে balance transfer হলো। A থেকে টাকা কাটা হলো কিন্তু B-তে credit হলো না। কোন ACID property দায়ী?

MODEL ANSWER — BB AD 2025 — ACID Scenario

উত্তর: Atomicity

ব্যাখ্যা:

Transaction ছিল দুটো operation:

Operation 1: Account A থেকে Debit (টাকা কাটা)

Operation 2: Account B-তে Credit (টাকা যোগ করা)

Operation 1 হয়েছে কিন্তু Operation 2 হয়নি — Transaction অর্ধেক হয়েছে।

Atomicity বলে: Transaction হয় সম্পূর্ণ হবে (দুটো operation-ই) অথবা কিছুই হবে না। Atomicity থাকলে যা হতো: Operation 2 fail করলে Operation 1-ও ROLLBACK হয়ে যেত। তখন Account A-তেও কোনো পরিবর্তন হতো না — টাকা হারাতো না। স্মরণে রাখো: "A to B transfer fails" = Atomicity সবসময়।

QUESTION: Other Bank 2024 Written: ACID Properties দিয়ে একটা database problem-এর case study solve করো।

MODEL ANSWER — ACID Properties — Banking Context Full Answer

একটা banking transaction scenario ধরো:

Customer Tk.10,000 withdraw করছে। Atomicity (সব-অথবা-কিছুই-না):

Transaction: (1) Account balance check, (2) Balance update, (3) Cash dispense, (4) Receipt print যদি Step 3-এ ATM jam হয় — Atomicity নিশ্চিত করে যে Step 2 (balance update) ROLLBACK হবে। Customer টাকা পাবে না কিন্তু balance কাটাও যাবে না। Consistency (সবসময় valid state): Rule: Balance < 0 হতে পারবে না। যদি current balance Tk.5,000 এবং customer Tk.10,000 তুলতে চায় — Consistency নিশ্চিত করে এই transaction reject হবে। Balance কখনো negative হবে না। Isolation (concurrent transaction-এ interference নেই): একসাথে দুটো ATM থেকে একই account-এ withdraw: T1 দেখছে balance = 10,000। T2-ও দেখছে balance = 10,000। Isolation থাকলে: T1 আগে complete হবে, balance = 0। T2 শুরু হবে updated balance দেখে — দ্বিতীয় withdrawal reject হবে। Isolation না থাকলে: দুটোই 10,000 দেখে দুটোই withdraw করতে পারে = 20,000 গেল! Durability (commit হলে permanent): Tk.10,000 withdraw successful হলো। Receipt দিল। Power গেল। Durability নিশ্চিত করে: Power আসার পরেও log দেখে এই transaction recorded আছে। Balance update undo হবে না।

05 TYPE 4 — ER Diagram Design

ER Diagram প্রস্নে text description দিয়ে বলা হয় diagram draw করতে। এখানে সবচেয়ে বড় চ্যালেঞ্জ: text পড়ে entity, attribute, এবং relationship বের করা।

Text থেকে ER Diagram বের করার Framework

কীভাবে ভাববেন: যেকোনো ER text পড়লে ৩ ধাপে ভাবো:

ধাপ ১ — ENTITY খোঁজো (Noun গুলো): "Soccer Club, Team, Player, Match, Referee" → এগুলো Entity (Rectangle)

ধাপ ২ — ATTRIBUTE খোঁজো (property গুলো): "Team-এর name, stadium, city আছে" → name, stadium, city হলো attribute (Oval)

ধাপ ৩ — RELATIONSHIP খোঁজো (Verb গুলো): "Player belongs to Team" → "belongs to" হলো relationship (Diamond) "Match is refereed by Referee" → "refereed by" হলো relationship

Cardinality বের করার Rule

Question করো...	উত্তর	Cardinality
"একটা Team-এ কতজন Player থাকতে পারে?"	অনেক (Many)	Team → Player: 1 to N
"একজন Player কতটি Team-এ খেলতে পারে?"	একটিতে (প্রস্নে বলা আছে "one club each")	Player → Team: N to 1
"একটা Match-এ কতটি Team?"	দুটো (home/guest)	Match → Team: N to M (two teams per match)
"একটা Match-এ কতজন Referee?"	তিনজন (প্রস্নে বলা আছে "exactly 3 referees")	Match → Referee: M to N

QUESTION: BB AP 2023: Token-ring LAN-এর class diagram: nodes (workstations, servers, printers), packets (originator, destination, content), circular configuration।

MODEL ANSWER — BB AP 2023 — Class Diagram Description

Entities/Classes: Node (abstract parent class)

Attributes: name (unique within network), nextNode (reference to next node) Child classes (Generalization/Inheritance): Workstation — originates messages (is_originator = true) Server — can receive messages Printer — can receive messages Packet Attributes: originator (Node), destination (Node), content (String) Methods: send(), receive() LAN Configuration: circular (ring) of Nodes Association: 1 LAN contains N Nodes Relationships: LAN 1 ----contains---- N Node Node 1 ----sends---- N Packet (Workstation only originates) Node 1 ----receives---- N Packet (Server, Printer receive) Node ----nextNode----> Node (self-association, circular) Cardinality: LAN to Node: 1:N (one LAN, many nodes) Workstation to Packet: 1:N (one workstation sends many packets) Packet to Node(destination): N:1 (many packets to one destination node)

PRACTICE: ER Practice: প্রতিদিন একটা text description পড়ো এবং নিজে ER diagram draw করো (কাগজে): ১. Library system: Books, Members, Loans, Authors ২. Hospital: Patients, Doctors, Appointments, Departments ৩. University: Students, Courses, Instructors, Enrollments Cardinality সবসময় দেখাও।

06 TYPE 5 — Short Note এবং Comparison

Short note প্রশ্নে তোমাকে একটা concept সংক্ষেপে লিখতে বলা হয়। এখানে structure সবচেয়ে গুরুত্বপূর্ণ।

Short Note-এর Template — সবসময় এই structure follow করো

সংজ্ঞা (Definition): [Concept] হলো...কীভাবে কাজ করে: এটা কাজ করে...উদাহরণ: Banking context-এ...সুবিধা/ব্যবহার: এটা ব্যবহার হয়...

QUESTION: Other Bank 2022 Written: What is a trigger? When is it used? Write advantages of triggers.

MODEL ANSWER — Trigger — Short Note

সংজ্ঞা:

Trigger হলো database-এ stored একটি special procedure যা automatically execute হয় যখন নির্দিষ্ট একটা event ঘটে — যেমন INSERT, UPDATE, বা DELETE। কীভাবে কাজ করে: Programmer trigger define করে দেয়: "যখন [event] ঘটে [table]-এ, তখন [action] করো।" Database সেই event detect করলে automatically trigger fire করে। Example: CREATE TRIGGER check_balance BEFORE UPDATE ON Account FOR EACH ROW BEGIN IF NEW.balance < 0 THEN SIGNAL SQLSTATE "45000" SET MESSAGE_TEXT = "Balance cannot be negative"; END IF; END; কখন ব্যবহার: Data validation — balance negative হওয়া prevent করা Audit trail — কে কখন কী change করল log করা Referential integrity enforce করা Automatic calculation — order total auto-update সুবিধা: Business rules automatically enforce হয় — manually check করতে হয় না Audit/log automatic হয় Data consistency নিশ্চিত হয় Application code ছাড়াই data validation হয়

QUESTION: Other Bank 2021: SQL vs NoSQL — comparison

Feature	SQL (Relational Database)	NoSQL (Non-Relational)
Structure	Fixed schema (table, row, col)	Flexible schema (document, key-value, graph)
Scaling	Vertical (bigger server)	Horizontal (more servers)
Query	SQL language (standardized)	Database-specific API
ACID	Full ACID compliance	Often BASE (eventual consistency)
Best for	Structured data, complex queries	Unstructured/semi-structured, high volume
Examples	MySQL, PostgreSQL, Oracle	MongoDB, Redis, Cassandra, Neo4j
Banking context	Transaction data (ACID important), account records, loan data	User session data, click logs, real-time analytics, social features

07 পরীক্ষায় Answer কীভাবে লিখবে

সঠিক answer জানা এবং সেটা mark পাওয়ার মতো করে লেখা — দুটো আলাদা skill। BB AD-এ partial marking আছে, তাই structure ঠিক থাকলে পুরোটা না পারলেও marks পাবে।

প্রতিটা Type-এর জন্য Answer Format

Question Type	Best Format	Examiner কী দেখে	Time (10 marks)
Normalization (Type 1)	ধাপ ১: FD list → ধাপ ২: PK → ধাপ ৩: NF check → ধাপ ৪: Decomposed tables	প্রতিটা ধাপ visible, FD correct, PK/FK marked	12-15 min
SQL Query (Type 2)	Query → ব্যাখ্যা (কেন এই logic)	Correct JOIN type, correct aggregate, correct condition	8-10 min
ACID Scenario (Type 3)	Property নাম → সংজ্ঞা → scenario-তে কীভাবে apply → কী হতো vs হয়েছে	Property correctly identified, scenario correctly explained	6-8 min
ER Diagram (Type 4)	Entities (boxes) → Attributes (ovals) → Relationships (diamonds) → Cardinality (numbers)	All entities, correct cardinality, PKs underlined	12-15 min
Short Note (Type 5)	Definition → How it works → Example → Advantage	Structured, concrete example, not just definition	5-7 min

পরীক্ষায় এই ভুলগুলো করো না

- **Normalization-এ:** FD না লিখে শুধু final table দেওয়া — FD লেখা mandatory, এখান থেকেই marks আসে
- **SQL-এ:** Syntax ভুল হলেও logic লেখো — examiner logic দেখে marks দেয়, syntax নয়
- **ACID-এ:** শুধু "Atomicity" লিখলেই হবে না — scenario-তে কীভাবে ভেঙেছে সেটা explain করতে হবে
- **ER-এ:** Diagram draw করলে cardinality (1, N, M) না লিখলে half marks কাটে
- **Short note-এ:** শুধু definition লিখলে হবে না — example দাও, তাহলে full marks আসবে

BB NOTE: BB AD Written-এ Partial Marking আছে। কোনো প্রশ্ন blank রাখবে না। FD বের করতে না পারলেও যা পেরেছ লেখো। SQL query ভুল হলেও structure লেখো। Normalization-এ পুরো decomposition না পারলেও যতটা পেরেছ লেখো। ০ mark-এর চেয়ে ৩-৪ mark সবসময় ভালো।

08 ১০ দিনের Practice Schedule

Theory পড়া এবং practice করা দুটো আলাদা। এই schedule follow করলে ১০ দিনে যেকোনো BB AD DBMS written question answer করতে পারবে।

দিন	Focus	কী করবে	সময়
Day 1	Normalization Concept	FD কী সেটা বোঝো। ৩টা ছোট schema নাও, FD manually বের করো।	1 hr
Day 2	1NF-2NF-3NF	BB AD 2025 schema normalize করো without looking at answer. তারপর check।	1.5 hr
Day 3	Normalization Practice	২টা নতুন schema নাও এবং নিজে step-by-step 3NF-এ নিয়ে যাও।	1.5 hr
Day 4	SQL Foundation	FROM-WHERE-GROUP BY-HAVING-SELECT order মুখস্থ করো। ৫টা basic query লেখো।	1 hr
Day 5	SQL JOIN	INNER, LEFT, RIGHT JOIN কখন কোনটা — ৩টা query each। BB AP 2023 romantic comedy solve করো।	1.5 hr
Day 6	SQL Advanced	GROUP BY + HAVING + subquery — ৫টা query from scratch। Other bank-এর SQL questions।	1.5 hr
Day 7	ACID Scenarios	৫টা ACID scenario পড়ো এবং কোন property বলো। BB AD 2025 MCQ explain করো।	1 hr
Day 8	ER Diagram	Library এবং Hospital system-এর ER diagram draw করো কাগজে।	1 hr
Day 9	Full Mixed Practice	Without notes: (a) Normalize একটা schema (b) Write 3 SQL queries (c) Explain one ACID scenario	1.5 hr
Day 10	Mock Test	BB AD DBMS-এর সব previous year written questions — timed 90 minutes	1.5 hr

APPROACH: সবচেয়ে গুরুত্বপূর্ণ দিন হলো Day 9 এবং Day 10। Notes না দেখে answer করা এবং নিজের mistake বের করা — এটাই real learning। Day 7 practice থেকে শেখা ভুলগুলো থেকে আরও বেশি শেখা হয় theory পড়া থেকে।

৬টা Trigger Word — দেখলেই বুঝবে কোন Type

Question-এ এই কথা দেখলে...	Type	প্রথমেই যা করবে
"normalize করো", "3NF-এ আনো", "schema decompose করো"	Type 1: Normalization	লিখো: FD: এবং সব dependencies বের করো
"SQL query লেখো", "find করো", "count করো", "UPDATE করো"	Type 2: SQL	লিখো: FROM (কোন table?) তারপর logic
"কোন ACID property?", "কোন property দায়ী?", "ACID explain"	Type 3: ACID	Scenario-তে কী ঘটেছে সেটা identify করো
"ER diagram draw করো", "entity-relationship design"	Type 4: ER Diagram	Text থেকে Noun (entity) এবং Verb (relationship) বের করো
"short note লেখো", "পার্থক্য বলো", "compare করো"	Type 5: Short Note	Structure: Definition → How it works → Example

সবচেয়ে গুরুত্বপূর্ণ mental shift:

Theory পড়ার পরপরই একটা নতুন scenario-তে apply করো।

"আমি জানি Normalization কী" এবং "আমি নতুন schema দেখলে normalize করতে পারব" — দুটো সম্পূর্ণ আলাদা। প্রতিটা concept পড়ার পরে ১টা নতুন problem solve করো। তাহলে BB AD Written তোমার কাছে সহজ হবে।